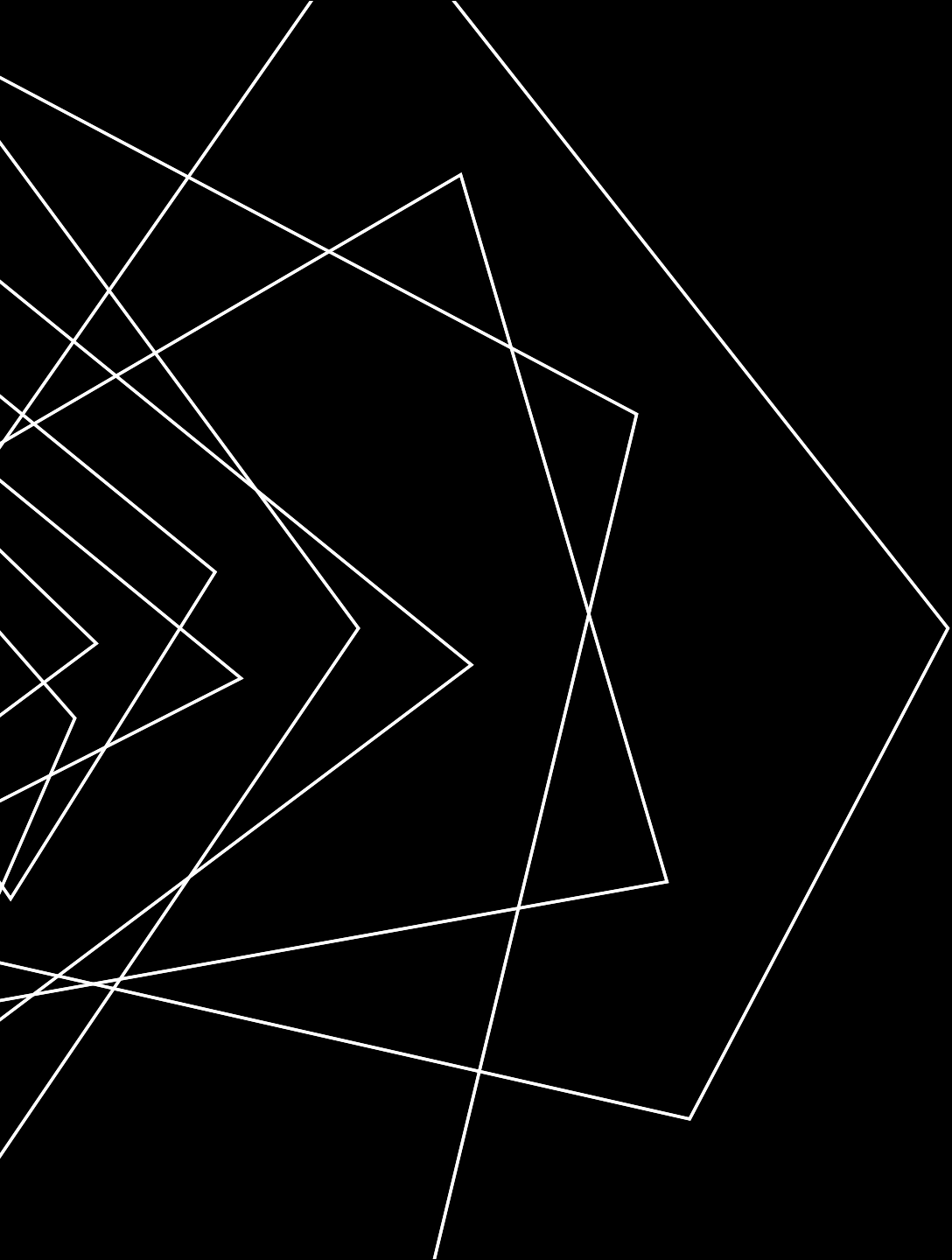




EAST TEXAS A&M
— UNIVERSITY —

CSCI 380.61E
WEB PROGRAMMING
AND INTERFACE DESIGN



ROUTING, VALIDATION & PLUGINS

Web Programming and Interface Design
- HAPIJS

LEARNING OBJECTIVES

- Understand Hapi routing and handlers.
- Use Joi for input validation.
- Organize app using plugins.

ROUTING IN HAPI.JS

- Define multiple routes for CRUD operations.
- Use route parameters and query strings.
- Example: `/users/{id}`, `/books?category=fiction`

HANDLING REQUESTS

- Access request payload with `request.payload`.
- Use POST and PUT for data submission.
- Respond using `h.response()` with status codes.

HAPI.JS ROUTING

- Routes: Define how your server responds to different HTTP requests.
- Route Structure:
 - Method: GET, POST, PUT, DELETE
 - Path: URL endpoint
 - Handler: Function to execute

API REQUEST METHODS

- GET: Retrieve data
- POST: Create new resource
- PUT: Update existing resource
- DELETE: Remove resource

HANDLING DIFFERENT HTTP METHODS

```
server.route([  
  {  
    method: 'GET',  
    path: '/items',  
    handler: (request, h) => {  
      return [{ id: 1, name: 'Item 1' }];  
    }  
  },  
],
```

HANDLING DIFFERENT HTTP METHODS

```
{  
  method: 'POST',  
  path: '/items',  
  handler: (request, h) => {  
    return { id: 2, name: request.payload.name };  
  },  
},
```

HANDLING DIFFERENT HTTP METHODS

```
{  
  method: 'PUT',  
  path: '/items/{id}',  
  handler: (request, h) => {  
    return { id: request.params.id, name:  
request.payload.name };  
  },  
},
```

HANDLING DIFFERENT HTTP METHODS

```
{  
  method: 'DELETE',  
  path: '/items/{id}',  
  handler: (request, h) => {  
    return h.response().code(204);  
  }  
}  
]);
```

CLOUD SECURITY FUNDAMENTALS

- Shared Responsibility Model:
 - Cloud Provider: Infrastructure security
 - Customer: Application security

COMMON SECURITY THREATS

- Data breaches
- Denial of Service (DoS) attacks
- Insider threats

INPUT VALIDATION

- Install Joi: `npm install @hapi/joi`
- Validate request payload using schemas.
- Prevent invalid data entry.

SECURITY IN HAPI.JS

- **Key Features:**

- Input validation
- Authentication and authorization

INPUT VALIDATION WITH JOI

```
const Joi = require('joi');
```

```
const schema = Joi.object({  
  name: Joi.string().min(3).required(),  
  email: Joi.string().email().required()  
});
```

INPUT VALIDATION WITH JOI

```
const Joi = require('joi');
```

```
const schema = Joi.object({  
  name: Joi.string().min(3).required(),  
  email: Joi.string().email().required()  
});
```

IMPLEMENTING AUTHENTICATION

- Using @hapi/cookie:
- `npm install @hapi/cookie`

SETTING UP COOKIE AUTHENTICATION

```
const cookieAuth = require('@hapi/cookie');
```

```
server.auth.strategy('session', 'cookie', {  
  cookie: {  
    name: 'session',  
    isSecure: false // Use true in production  
  },  
  redirectTo: '/login'  
});
```

SETTING UP COOKIE AUTHENTICATION

```
server.route({  
  method: 'GET',  
  path: '/login',  
  handler: (request, h) => {  
    return 'Login Page';  
  }  
});
```

ADVANCED SECURITY MEASURES

- API Security Best Practices
- Encryption and Data Protection

API SECURITY BEST PRACTICES

- Key Practices:
 - Use HTTPS
 - Implement rate limiting
 - Validate user input

JWT AUTHENTICATION

- Overview: JSON Web Tokens for secure API communication.
- Installation: `npm install @hapi/jwt`

SETTING UP JWT AUTHENTICATION

```
const Jwt = require('@hapi/jwt');

server.auth.strategy('jwt', 'jwt', {
  keys: 'your_secret_key',
  validate: (decoded, request) => {
    return { isValid: true }; // Add your validation logic here
  }
});
```

SETTING UP JWT AUTHENTICATION

```
server.auth.default('jwt');
```

```
server.route({  
  method: 'GET',  
  path: '/protected',  
  handler: (request, h) => {  
    return 'This is a protected route';  
  }  
});
```

SECURING API ENDPOINTS

```
server.route({  
  method: 'GET',  
  path: '/users',  
  options: {  
    auth: 'jwt' // Protect this endpoint  
  },  
  handler: (request, h) => {  
    return 'User data';  
  }  
});
```

SECURING API ENDPOINTS

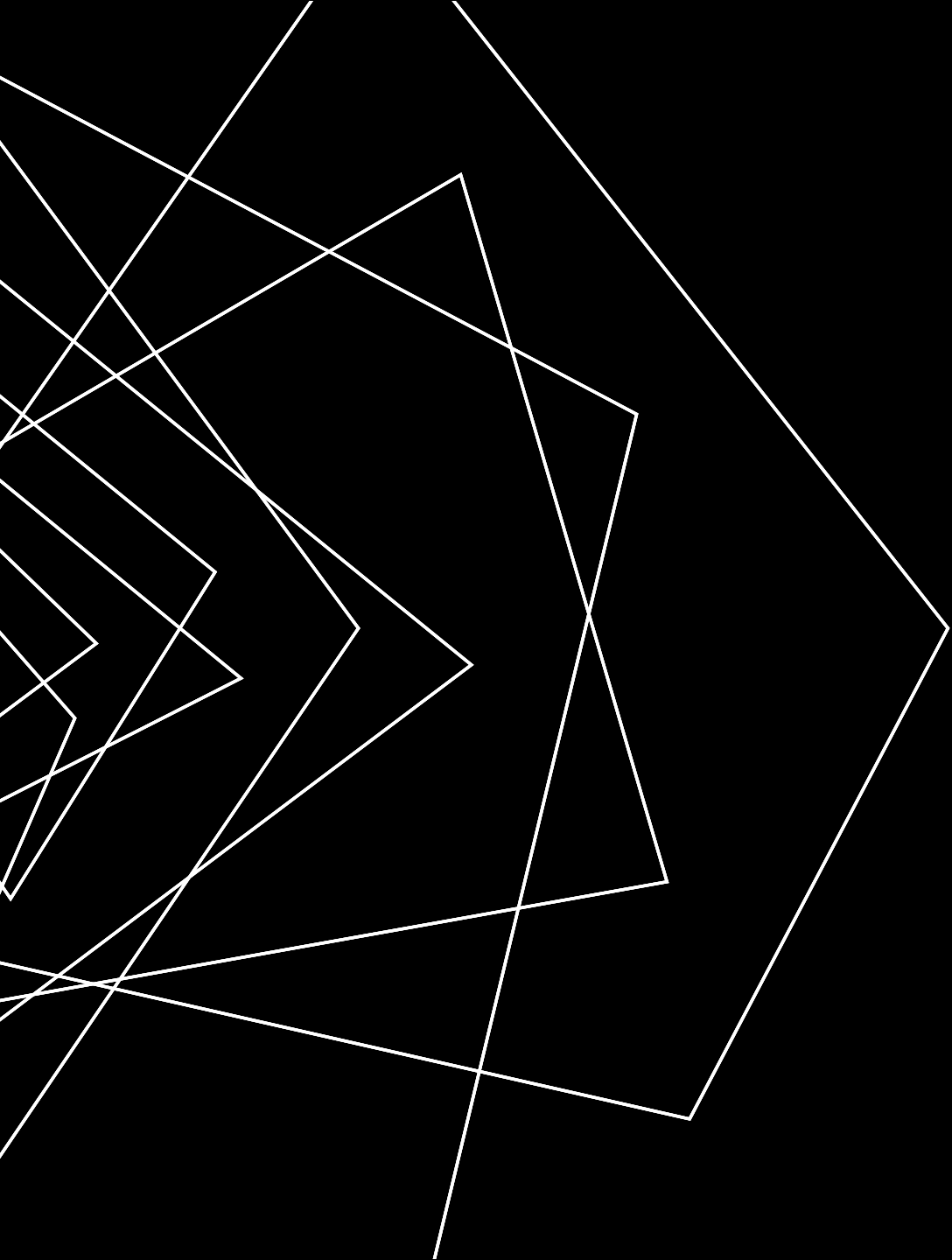
```
server.route({  
  method: 'GET',  
  path: '/users',  
  options: {  
    auth: 'jwt' // Protect this endpoint  
  },  
  handler: (request, h) => {  
    return 'User data';  
  }  
});
```

PLUGINS

- Hapi plugins modularize routes and logic.
- Example: `server.register(require('./routes/users'));`
- Encourages scalable project structure.

LAB

- Create CRUD API for a resource (Books, Users).
- Implement Joi validation for POST and PUT requests. (Optional)



THANK YOU

Yau Ka Cheung
Yauka.Cheung@etamu.edu
ACB1 Room 335